Research Intake 2026

Day 13

WORD EMBEDDINGS
&
VECTOR REPRESENTATIONS

A Beginner's Guide for Students

From foundational concepts to research-level understanding

Gudsky Research Foundation

Table of Contents

1 Limitations of Discrete Representations

Why BoW and TF-IDF leave meaning on the table

The Bag-of-Words model and its TF-IDF extension encode each word as a single position in a vocabulary-sized vector space. In a vocabulary of 50,000 words, 'car' is represented as a vector with a 1 in position 3,241 and zeros everywhere else; 'automobile' has a 1 in position 4,872 and zeros everywhere else. These two vectors are perfectly orthogonal — their dot product is exactly zero — which means the model has encoded the claim that 'car' and 'automobile' share no common properties whatsoever. A classifier trained on documents about cars using the word 'automobile' has learned nothing that helps it classify new documents about cars that use the word 'car'. The semantic relationship that is obvious to any human reader is entirely invisible to the model.

This problem is not a minor inconvenience: it is a fundamental architectural limitation that cascades through every downstream task. In information retrieval, a query for 'vehicle accident' will fail to retrieve highly relevant documents that use the words 'car crash' instead, because no mechanism exists to recognise the near-synonymy of 'vehicle' with 'car' and 'accident' with 'crash'. In sentiment analysis, the model treats 'excellent', 'outstanding', 'brilliant', and 'superb' as four completely unrelated tokens, requiring independent statistical evidence for each when their sentiment valences could in principle be shared. In machine translation, the correspondence between words in different languages — which is precisely the information a translation system needs — is entirely absent from discrete representations.

The sparsity problem compounds the orthogonality problem. In a typical document classification corpus, any given document contains at most a few hundred of the 50,000 vocabulary tokens; the remaining 49,700+ dimensions of its representation are zero. This extreme sparsity means that any two documents have very small overlap in their non-zero features, making similarity computation unreliable. The cosine similarity between two topically related documents may be nearly zero simply because they used different vocabulary to express the same concepts. Dense vector representations, which assign real-valued coordinates in a compact 100-to-300-dimensional space to every word, address both the orthogonality and sparsity problems simultaneously.

Property	One-Hot / BoW Encoding	Dense Embedding
Dimensionality	:50,000 (vocabulary size)	50–300 (fixed, learned)
Semantic similarity	None — all words orthogonal	Yes — cosine similarity meaningful
Analogical reasoning	Impossible	Yes — vector arithmetic works
Memory per vocabulary	$O(\|V\|^2)$ for pairwise similarities	$O(\|V\| \times d)$ — linear in vocab size
OOV handling	Impossible — new words = unknown	Via subwords (fastText) or averaging
Training signal	Explicit labels required	Self-supervised from raw text

2 Distributional Hypothesis

The linguistic foundation of word vectors

The distributional hypothesis, first articulated by linguist John Rupert Firth in 1957 with the aphorism 'You shall know a word by the company it keeps', provides the theoretical foundation for every word embedding method developed over the subsequent seven decades. The hypothesis states that words appearing in similar linguistic contexts tend to have similar meanings. This is an empirical claim about the statistical structure of language, not a definition of meaning — it says that context similarity is a reliable proxy for semantic similarity, not that context identity is semantic identity.

The linguistic evidence for the distributional hypothesis is extensive and comes from multiple sources. Substitution tests — the standard linguistics technique of asking whether replacing word A with word B produces a grammatical and semantically coherent sentence — show that words that pass the substitution test for each other (i.e., near-synonyms) occur in systematically similar contexts. Acquisition studies show that children learn word meanings primarily from the contexts in which they hear words used, not from explicit definitions. Lexicographers construct dictionary definitions by analysing the contexts in which words appear. The fact that NLP systems can successfully approximate synonym relationships, semantic role assignments, and even commonsense reasoning from distributional statistics strongly suggests that the distributional structure of text encodes a substantial fraction of semantic knowledge.

The formal implementation of the distributional hypothesis begins with a co-occurrence matrix: a matrix where entry (i,j) records how many times word i appeared within a fixed context window of word j across a large corpus. Words with similar meanings will have similar rows in this matrix because they appear with similar context words. 'Doctor' and 'physician', for example, both appear frequently near 'hospital', 'patient', 'diagnosis', 'treatment', and 'prescription', giving their rows high correlation. The challenge is that this raw co-occurrence matrix is enormous (vocabulary size squared) and sparse; the dimensionality reduction methods developed to address this challenge — from Latent Semantic Analysis to Word2Vec — are the substance of this session.

Latent Semantic Analysis (Deerwester et al., 1988) — The First Large-Scale Distributional Model

LSA applied Singular Value Decomposition (SVD) to the word-document co-occurrence matrix, projecting words into a dense low-dimensional space where semantically related words cluster together. It dramatically improved information retrieval by enabling 'car' queries to retrieve 'automobile' documents. LSA demonstrated that the distributional hypothesis could be operationalised at scale using linear algebra — a conceptual breakthrough that established the mathematical framework Word2Vec and GloVe would later extend with neural training objectives.

The distributional hypothesis has both explanatory power and known limitations. It explains why distributional methods succeed at capturing thematic and associative similarity — 'coffee' and 'tea' appear in similar contexts and end up near each other in embedding space. It is less successful at capturing complementary or contrastive relationships: 'hot' and 'cold' appear in very similar contexts (both occur near 'temperature', 'weather', 'drink') and may be relatively close in embedding space despite being antonyms. The hypothesis also cannot explain semantic relationships that are not reflected in typical usage patterns: the fact that whales are mammals is not reliably recoverable from co-occurrence statistics alone, because texts about whales rarely also discuss mammalian biology. These limitations motivate the ongoing research into knowledge-grounded and commonsense-aware embeddings.

3 Word2Vec

Neural word embeddings at scale

Word2Vec (Mikolov et al., Google, 2013) is the landmark paper that transformed word embeddings from a specialised research technique into a mainstream NLP tool. The key innovation was not the idea of distributed word representations — those had existed since Hinton's connectionist models in the 1980s and Bengio's neural language model in 2003 — but the training efficiency that made it practical to learn high-quality embeddings from billions of words in a matter of hours on a single machine. By eliminating the expensive hidden layer of previous neural language models and replacing the full softmax with the negative sampling approximation, Word2Vec made large-scale embedding training accessible to the entire NLP community.

Word2Vec introduces two architectures — CBOW and Skip-Gram — that implement the distributional hypothesis as a prediction task: the network is trained to predict words from their contexts (or contexts from words), and the internal representations learned by the network to perform this prediction become the embeddings. The prediction task is entirely self-supervised: no human annotation is required, only raw text. This means embeddings can be trained on any available text corpus, and larger, more diverse corpora generally produce better embeddings. The Google News embeddings distributed with the original paper were trained on 100 billion words, a scale that was unprecedented in NLP at the time.

3.1 CBOW Architecture

Continuous Bag of Words (CBOW) implements the prediction problem: given the context words surrounding a target word, predict the target word. For the sentence 'The cat sat on the mat' with a window size of 2, CBOW would construct training examples where the context words ['The', 'cat', 'on', 'the'] are used to predict the target word 'sat'. The 'bag of words' component of the name refers to the fact that context words are combined by averaging their embedding vectors, discarding their relative positions. The averaged context vector is passed through a single linear transformation and a softmax layer over the vocabulary to produce a probability distribution over potential target words; the training objective minimises the cross-entropy loss between this distribution and the one-hot encoding of the true target word.

The CBOW architecture has a specific performance profile that makes it preferable in certain settings. By averaging multiple context word vectors before making a prediction, CBOW performs a kind of implicit smoothing that reduces sensitivity to individual word occurrences. This smoothing effect improves performance on frequent words, which appear in many different contexts whose averaging produces stable training signals. CBOW also trains faster than Skip-Gram for the same corpus because each training step uses all context words to make a single prediction, while Skip-Gram makes one prediction per context word. For large corpora where frequent-word quality is the priority, CBOW is the preferred architecture.

3.2 Skip-Gram Architecture

Skip-Gram inverts the CBOW prediction problem: given a single centre word, predict each of the surrounding context words individually. For the centre word 'sat' with a window of 2, Skip-Gram generates four separate training pairs: (sat, The), (sat, cat), (sat, on), (sat, the). Each pair is treated as an independent training example — the model is trained to predict that 'The', 'cat', 'on', and 'the' are all legitimate neighbours of 'sat', and to assign low probability to random vocabulary words that are not typical neighbours. This one-to-many expansion means Skip-Gram generates many more training examples per word than CBOW, which is why it trains more slowly.

The performance advantage of Skip-Gram over CBOW becomes apparent for infrequent and domain-specific vocabulary. An infrequent word like 'syzygy' (the alignment of three celestial bodies) might appear only a few hundred times in a large corpus. Under CBOW, those few hundred occurrences generate only a few hundred training examples where 'syzygy' is the prediction target. Under Skip-Gram, each occurrence generates several training examples — one for each context word — multiplying the training signal by the

window size. This amplification of the training signal for rare words is the primary reason Skip-Gram produces better embeddings for domain-specific and technical vocabulary, making it the preferred architecture for scientific, legal, and medical NLP applications.

Word2Vec — Architectural Comparison

CBOW (Continuous Bag of Words):
  Input : context words ['The', 'cat', __, 'on', 'the']
  Process: average context embeddings → linear layer → softmax
  Output : predict centre word 'sat'
  Best for: frequent words, fast training

Skip-Gram:
  Input : centre word 'sat'
  Process: embedding lookup → linear layer → softmax per context
  Output : predict each context word → ('sat','The'), ('sat','cat'), ...
  Best for: rare/domain words, richer embeddings

Window size w=2 on 'The cat sat on the mat':
  CBOW → 1 training example per centre word
  Skip-Gram → 2×w = 4 training examples per centre word

3.3 Negative Sampling

The training objective of predicting a word over a vocabulary of 50,000 entries requires computing a softmax over the entire vocabulary at every training step. This operation involves a matrix multiplication of dimension d × |V| (where d is the embedding dimension and |V| is the vocabulary size) plus a normalisation over all 50,000 logits. For a typical embedding dimension of 300 and vocabulary size of 50,000, this is a computation of 15 million multiply-add operations per training example — repeated millions of times per epoch. On hardware available in 2013, this made full-vocabulary Word2Vec training on billion-word corpora prohibitively slow.

Negative sampling (Mikolov et al., 2013, inspired by Noise Contrastive Estimation) approximates the full softmax with a binary classification problem that is many orders of magnitude cheaper to compute. For each positive training pair (centre word, true context word), the model samples k negative words — typically 5 to 20 — drawn randomly from the vocabulary according to a modified unigram distribution (specifically the unigram distribution raised to the power 3/4, which slightly upweights rare words to ensure they receive adequate training signal). The training objective becomes: maximise the probability that the true context word is a genuine neighbour of the centre word, and minimise the probability that each of the k noise words is a genuine neighbour. This reduces computation from O(|V|) to O(k) per training step — a reduction of 1,000× or more for typical vocabulary sizes.

The theoretical justification for negative sampling connects to the broader framework of noise contrastive estimation. By framing word prediction as a discrimination task between real and fake context words, the model is implicitly learning to model the ratio of the probability of the true context to the probability of a random noise word. Levy and Goldberg (2014) later proved that Skip-Gram with Negative Sampling is mathematically equivalent to factorising a matrix of Shifted Positive Pointwise Mutual Information (SPPMI) values — a result that unified the prediction-based (Word2Vec) and count-based (LSA, GloVe) traditions and provided a theoretical explanation for why the learned embeddings capture the statistical structure of word co-occurrence.

3.4 Vector Arithmetic: Analogy Reasoning

The most celebrated property of Word2Vec embeddings is their capacity for semantic analogy reasoning via simple vector arithmetic. The canonical example — vector('king') − vector('man') + vector('woman') ≈

vector('queen') — was surprising to the NLP community because nothing in the training objective explicitly required or encouraged this behaviour. The model was trained to predict context words, not to represent analogical relationships; the analogical structure emerged as an implicit consequence of the distributional statistics of the training corpus.

The geometric interpretation of analogy arithmetic is that semantic relationships are encoded as consistent translation vectors in embedding space. The 'gender direction' — the vector from any masculine concept to its feminine equivalent — is approximately constant across many pairs: vector('king') − vector('queen') ≈ vector('man') − vector('woman') ≈ vector('actor') − vector('actress') ≈ vector('uncle') − vector('aunt'). Similarly, the 'capital-country direction' is approximately constant: vector('Paris') − vector('France') ≈ vector('Berlin') − vector('Germany') ≈ vector('Tokyo') − vector('Japan'). These consistent directional offsets imply that the embedding space has learned to organise semantic relationships as geometric transformations — a striking and unexpected form of geometric structure.

The practical applications of analogy arithmetic extend beyond the demonstration examples. In information retrieval, analogy queries can find domain-specific technical term correspondences: given 'aspirin is to headache as X is to arthritis', the arithmetic retrieves 'ibuprofen' or 'naproxen'. In multilingual NLP, the analogy structure of embedding spaces from different languages is approximately isomorphic — a finding that motivates cross-lingual embedding alignment methods where a rotation matrix is learned to map embeddings from one language space to another, enabling zero-shot cross-lingual transfer for NER and classification tasks.

Research Spotlight: Linguistic Regularities in Continuous Space Word Representations (Mikolov et al., 2013)

This paper showed that vector arithmetic captured not just semantic but syntactic regularities: vector('running') − vector('run') ≈ vector('swimming') − vector('swim'); vector('biggest') − vector('big') ≈ vector('coldest') − vector('cold'). The paper introduced the 3CosAdd evaluation metric for analogy tasks and the Google Analogy Test Set (8,869 semantic + 10,675 syntactic analogy questions), which became the standard benchmark for evaluating word embeddings for the following decade.

4 GloVe: Global Vectors

Count-based meets neural

GloVe (Global Vectors for Word Representation, Pennington et al., Stanford NLP Group, 2014) was developed as a principled alternative to Word2Vec that explicitly leverages global corpus statistics rather than processing text through a local sliding window. The central observation motivating GloVe is that the ratio of co-occurrence probabilities between two words and a probe word carries more semantic information than the raw co-occurrence counts: the ratio P('ice' | 'solid') / P('steam' | 'solid') is large (ice is solid, steam is not), while P('ice' | 'water') / P('steam' | 'water') is near 1 (both ice and steam relate to water). This ratio structure — not captured by Word2Vec's local window approach — is what GloVe's objective function is designed to encode in the dot products of word vectors.

4.1 Co-occurrence Matrix

GloVe begins by constructing the global word-word co-occurrence matrix X across the entire corpus. Entry X_{ij} records how many times word j appeared in the context window of word i across all occurrences of word i in the training corpus. The context window is typically symmetric (extending w words to the left and right of the target word) and often uses a distance-weighted scheme where context words at distance d receive weight 1/d rather than the same weight regardless of distance — reflecting the linguistic intuition that immediately adjacent words are stronger indicators of semantic association than words several positions away.

The computational requirements for constructing this matrix distinguish GloVe from Word2Vec in a practically important way. For a vocabulary of 400,000 words (the size of the GloVe Wikipedia+Gigaword pre-trained model), the full co-occurrence matrix has 400,000 × 400,000 = 160 billion entries. Even stored as a sparse matrix (only non-zero entries), this requires processing the entire corpus once to count all co-occurrences, then holding the sparse matrix in memory or on disk during training. For the Wikipedia + Gigaword 5 corpus (6 billion tokens), GloVe training required approximately 40 CPU-hours for 300-dimensional vectors — substantially more infrastructure than the streaming Word2Vec approach, which requires only a single pass through the data.

4.2 GloVe Loss Function

The GloVe objective function is derived from the requirement that the dot product of word vectors w_i and context vectors $\tilde{w}_j$ should approximate the log of their co-occurrence count log(X_{ij}). The raw formulation — minimising $(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$ over all word pairs — has a critical problem: log(0) is undefined for word pairs that never co-occur, and these zero-count pairs numerically dominate the training set because the co-occurrence matrix is extremely sparse. GloVe addresses this by restricting the sum to word pairs with non-zero co-occurrence counts and introducing a weighting function f(X_{ij}) that down-weights very frequent co-occurrences.

GloVe Loss Function — Formal Definition

$$J = \Sigma_{i,j} f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

where:
- w_i = word vector for word i
- $\tilde{w}_j$ = context vector for word j (separate parameter set)
- b_i, $\tilde{b}_j$ = bias terms
- X_{ij} = co-occurrence count of word j in context of word i

Weighting function $f(x) = (x/x_{max})^{\alpha}$ if $x < x_{max}$, else 1
- $x_{max} = 100$ (cap on very frequent pairs)
- $\alpha = 0.75$ (sub-linear downweighting)

Final embedding for word i = w_i + w̃_i (sum of both parameter sets)

The weighting function f(X_{ij}) serves two purposes. First, it caps the contribution of very frequent word pairs: function words like 'the' and 'of' co-occur with almost every content word in the vocabulary; without down-weighting, the training signal would be dominated by these high-frequency pairs whose co-occurrence is largely uninformative about specific word meanings. Second, the sub-linear weighting with $\alpha = 0.75$ (rather than a hard cut-off at x_max) creates a smooth interpolation that prevents abrupt discontinuities in the training objective. The sum of the word vector w_i and its corresponding context vector w̃_i — two separate learned parameters for each word in GloVe's formulation — serves as the final embedding used in downstream tasks, averaging out the asymmetry between the two sets of parameters.

4.3 Word2Vec vs GloVe in Practice

The empirical comparison between Word2Vec and GloVe has been extensively studied since both models were released in 2013–2014. On the standard word similarity benchmarks (WordSim-353, SimLex-999, MEN) and the Google Analogy Test Set, GloVe and Word2Vec Skip-Gram perform comparably, with neither consistently dominating across all datasets and evaluation conditions. The performance gap between them is typically smaller than the performance gap attributable to differences in training corpus size, corpus quality, and hyperparameter settings — suggesting that the choice between them is less important than the choice of training data.

GloVe has a practical advantage in training efficiency once the co-occurrence matrix has been constructed: because GloVe minimises a weighted least-squares objective over a fixed matrix rather than processing streaming text examples, it is amenable to parallelisation and can exploit the structure of the factorisation problem more directly. Word2Vec, by contrast, processes training examples sequentially with stochastic gradient descent, which is inherently more sequential. For very large corpora (hundreds of billions of tokens), Word2Vec's streaming approach avoids the memory bottleneck of constructing and storing the co-occurrence matrix, giving it a practical advantage at extreme scale. fastText consistently outperforms both methods when out-of-vocabulary generalisation and morphological robustness are important, largely superseding them for new projects.

5 fastText: Subword Embeddings

Character n-grams for morphological robustness and OOV generalisation

fastText (Bojanowski et al., Facebook AI Research, 2017) extends the Word2Vec Skip-Gram model with a crucial modification: rather than assigning a single embedding vector to each word token, fastText represents each word as the sum of embedding vectors for all of its character n-grams. The word 'apple' (with added boundary markers '<apple>') is decomposed into the n-grams '<ap', 'app', 'ppl', 'ple', 'le>' (using n=3) plus the whole-word special token '<apple>'. The embedding for 'apple' is then the sum (or average) of the individual embedding vectors for each of these n-gram units. This decomposition is performed dynamically at training and inference time, requiring only the n-gram vocabulary to be stored rather than the full word vocabulary.

The architectural consequence of this subword decomposition is that fastText can generate an embedding for any word encountered at inference time, including words that were never seen during training. When a user query or input document contains a misspelling, a new technical term, a brand name, or a morphologically complex form not present in the training vocabulary, fastText composes an embedding from the character n-grams of that new word. The composed embedding will be most similar to the embeddings of training words that share the most character n-grams, which in practice means morphologically related words. 'Unhappiest', even if never seen during training, will be represented by an embedding close to those of 'unhappy', 'happiest', 'happiness', and 'unhappiness', which all share many character n-grams.

The practical benefits of this approach are most pronounced for morphologically rich languages and noisy text domains. For agglutinative languages like Turkish, Finnish, and Hungarian — where words are formed by chaining suffixes onto roots, potentially producing hundreds of inflected forms per lexeme — Word2Vec and GloVe assign an independent vector to each inflected form, requiring independent training evidence for each. A Turkish corpus of a given size may contain each inflected form only a small number of times, producing poorly estimated embeddings for rare forms. fastText shares the embedding components (the character n-grams corresponding to the shared root and individual suffixes) across all inflected forms, pooling statistical evidence across the morphological paradigm and producing better-estimated embeddings even for forms seen only a handful of times.

> 💡 **Use Case: Morphologically Rich Languages and Social Media NLP**
>
> For Turkish, Finnish, or Hungarian (which form words by chaining suffixes), Word2Vec assigns unique vectors to each inflected form, requiring independent statistical evidence for each. fastText shares subword embeddings across inflections, dramatically improving performance with smaller training corpora. In social media NLP, fastText's character n-gram approach handles misspellings ('recieve' shares n-grams with 'receive'), phonetic spellings ('luv' with 'love'), and neologisms ('selfie' can be partially represented even if unseen). The fastText library provides pre-trained embeddings for 157 languages, many of which lack sufficient data for high-quality word-level embeddings.

fastText also introduced a highly efficient text classification model — distinct from the embedding component — that averages word (or n-gram) representations and passes them through a linear classifier. Despite its simplicity, this classifier achieves accuracy competitive with deep learning models on many short-text classification benchmarks while running orders of magnitude faster. The fastText classification model is trained in seconds on standard hardware for datasets of millions of documents, making it the standard baseline for text classification tasks and the default choice for production systems with strict latency constraints. Facebook Research distributes both pre-trained word vectors and the fastText classification framework as open-source software.

6 Contextualised Embeddings

ELMo and the bridge to transformers

Word2Vec, GloVe, and fastText share a fundamental limitation: they produce static embeddings, assigning exactly one vector to each word type regardless of the sentence context in which it appears. The word 'bank' receives the same vector whether it appears in 'the river bank was eroded' or 'the central bank raised interest rates', even though these two uses of 'bank' refer to semantically distinct entities. Human language understanding, by contrast, is deeply contextual: the interpretation of every word is shaped by the words surrounding it, the broader discourse context, and world knowledge about the situations being described. Static embeddings can only represent the average meaning of a word type across all its uses, which may not correspond well to any specific use.

The polysemy problem — the fact that most content words in natural language have multiple distinct senses — is particularly acute for high-frequency words. The 100 most common English content words each have, on average, more than 10 distinct senses in WordNet. For any word with multiple senses, the static embedding represents a weighted average of those senses, with the weighting determined by the relative frequency of each sense in the training corpus. This means the embedding for 'bank' is dominated by the financial institution sense (which is far more frequent in typical news corpora) and provides poor representation for the 'river bank' or 'blood bank' senses that appear in other contexts. A downstream model that must distinguish between these senses receives a feature vector that conflates them.

ELMo (Embeddings from Language Models, Peters et al., Allen Institute for AI, 2018) was the first widely adopted solution to the contextualisation problem. ELMo trains a deep bidirectional LSTM language model on a large text corpus (1 billion words in the original paper) to predict each word from all preceding words (forward LSTM) and from all following words (backward LSTM). The concatenation of the forward and backward hidden states at each layer, combined across all layers of the LSTM stack, forms the contextualised representation for each word position. Unlike Word2Vec, which produces the same vector for 'bank' regardless of context, ELMo produces a different vector for each occurrence of 'bank' based on the full sentence context.

The empirical impact of ELMo was striking and immediate. Peters et al. (2018) demonstrated that replacing static GloVe embeddings with ELMo representations improved state-of-the-art performance on six major NLP benchmarks simultaneously: question answering (SQuAD), textual entailment (SNLI), semantic role labelling, coreference resolution, named entity recognition, and sentiment analysis. These improvements, ranging from 3% to 20% on individual tasks, established contextualised representations as the dominant paradigm in NLP and directly set the stage for BERT. ELMo is best understood as the proof of concept that contextualisation was worth the added computational cost — a finding that motivated the subsequent investment in transformer-based pre-training.

ELMo vs Static Embeddings — Representation Comparison

Static (Word2Vec/GloVe):
  'bank' in 'river bank' → [0.25, -0.73, 0.11, ...] (same vector)
  'bank' in 'central bank' → [0.25, -0.73, 0.11, ...] (same vector)
  Conflates all senses into one average representation

ELMo (Deep Bidirectional LSTM):
  'bank' in 'river bank eroded' → [0.87, 0.12, -0.44, ...] (river/geology context)
  'bank' in 'central bank rate' → [-0.31, 0.95, 0.63, ...] (finance context)
  Different vector per occurrence — context-sensitive representation

ELMo architecture: 2-layer biLSTM + character CNN for token encoding
  Layer 0 = character CNN (morphological / surface features)

```
Layer 1 = biLSTM-1       (syntactic features dominate)
Layer 2 = biLSTM-2       (semantic features dominate)
Final representation = learned weighted sum of all layers
```

ELMo's multi-layer architecture has a second important property beyond contextualisation: different layers of the network specialise in different types of linguistic information. Probing experiments — where small classifiers are trained to predict specific linguistic labels (POS tags, dependency labels, semantic roles) from the hidden states of each layer — revealed that lower LSTM layers capture predominantly syntactic information while upper layers capture semantic and task-specific information. This stratification of linguistic knowledge across network depth was subsequently found to hold for transformer models as well, and has important implications for transfer learning: the appropriate layer to use for a downstream task depends on whether that task is primarily syntactic or semantic in nature.

7 Practical Guide: Loading and Using Pre-trained Embeddings

Formats, integration patterns, and engineering considerations

Pre-trained word embeddings represent hundreds of GPU-hours of computation on massive text corpora. For most NLP projects, loading a high-quality pre-trained embedding is far more effective than training embeddings from scratch on a small domain corpus. The practical decisions involved in using pre-trained embeddings — which model to choose, how to load and integrate it, whether to freeze or fine-tune the embedding layer — significantly impact both model performance and engineering complexity.

The selection of a pre-trained embedding model should be driven by three factors: the domain of the target task relative to the embedding's training corpus, the language(s) involved, and the downstream model architecture. For English general-domain tasks, Google News Word2Vec (100B words, 300 dimensions), GloVe Common Crawl (840B tokens, 300 dimensions), or fastText Common Crawl (600B tokens, 300 dimensions) are the standard choices, with fastText preferred when OOV robustness is important. For domain-specific tasks, models pre-trained on domain text — BioWordVec for biomedical NLP, FinBERT embeddings for financial text, LegalBERT for legal documents — consistently outperform general-domain embeddings. For multilingual tasks, fastText provides pre-trained vectors for 157 languages, and multilingual transformer models (mBERT, XLM-R) provide contextualised representations across 100+ languages.

Pre-trained embeddings are distributed in three standard file formats, each with different loading procedures and use cases. Binary format (.bin files in the Word2Vec format) store embeddings as raw binary floating-point arrays with a header specifying vocabulary size and embedding dimension. These are loaded most efficiently with Gensim's KeyedVectors.load_word2vec_format(path, binary=True) method. Text format (.txt or .vec files) store one word and its space-separated float values per line, human-readable but larger on disk; loaded with KeyedVectors.load_word2vec_format(path, binary=False) or by parsing manually. GloVe files use a simplified text format without the header line of Word2Vec format and require the binary=False flag with no_header=True. Hugging Face's transformers library provides a unified interface for all modern transformer-based embeddings through the AutoModel and AutoTokenizer API.

Embedding Matrix Initialisation — PyTorch and Keras

```
# PyTorch: load pre-trained embeddings into nn.Embedding layer
from gensim.models import KeyedVectors
import torch, numpy as np

wv = KeyedVectors.load_word2vec_format('GoogleNews-vectors.bin', binary=True)
vocab_size, embed_dim = len(word2idx), 300
embedding_matrix = np.zeros((vocab_size, embed_dim))
for word, idx in word2idx.items():
    if word in wv:  embedding_matrix[idx] = wv[word]
    # OOV words remain zero-initialized

embedding_layer = torch.nn.Embedding(vocab_size, embed_dim)
embedding_layer.weight.data.copy_(torch.from_numpy(embedding_matrix))
embedding_layer.weight.requires_grad = False  # freeze: True to fine-tune

# Keras equivalent:
Embedding(vocab_size, embed_dim, weights=[embedding_matrix],
          input_length=max_len, trainable=False)
```

The decision to freeze or fine-tune the embedding layer is one of the most consequential hyperparameters in an NLP model. Freezing (setting requires_grad=False in PyTorch or trainable=False in Keras) prevents the pre-trained embedding vectors from being updated during training. This is appropriate when the target

domain closely matches the pre-training domain, the training dataset is small (fine-tuning with insufficient data can corrupt the pre-trained representations), and training speed is a priority. Fine-tuning allows the embeddings to be adapted to the target domain and task, improving performance when the domain is specialised, the training dataset is large, or the vocabulary of the target domain is substantially different from the pre-training vocabulary. A common compromise is to initially freeze the embeddings for the first few training epochs (allowing the upper layers to initialise to reasonable values) and then unfreeze them for the remainder of training with a small learning rate.

Model	Training Corpus	Dimensions	Best For
Word2Vec (Google)	Google News, 100B words	300	News, general English NLP
GloVe (Stanford)	Common Crawl, 840B tokens	50/100/200/300	Multiple granularities; well-studied
fastText (Facebook)	Common Crawl + Wikipedia, 157 langs	300	Multilingual, morphological, OOV robustness
BioWordVec	PubMed + MIMIC-III clinical notes	200	Biomedical and clinical NLP
ELMo (AllenAI)	1 Billion Word Benchmark	1024 (contextual)	Tasks requiring word sense disambiguation
BERT (Google)	Wikipedia + Books, 3.3B tokens	768/1024 (contextual)	Most downstream tasks; fine-tuning baseline

8 Bias in Word Embeddings

Societal stereotypes encoded in vector space

Word embeddings trained on large text corpora faithfully reproduce the statistical patterns of that text — including the patterns that reflect societal biases, historical inequities, and discriminatory associations. Bolukbasi et al. (2016) provided the first systematic quantitative analysis of gender bias in Word2Vec embeddings trained on Google News. Using the analogy evaluation framework developed by Mikolov et al., they demonstrated that the embedding space encodes gender stereotypes as consistent geometric relationships: 'man is to doctor as woman is to nurse', 'man is to computer programmer as woman is to homemaker', 'man is to philosopher as woman is to housewife'. These are not edge cases or artefacts; they are stable, reproducible geometric relationships that emerge from the distributional statistics of a corpus that reflects decades of gender-biased language use in professional and public discourse.

The mechanism by which bias enters embeddings is the same mechanism by which any information enters embeddings: through co-occurrence statistics. If news articles about doctors more frequently use male pronouns, and news articles about nurses more frequently use female pronouns, then 'doctor' will co-occur more with the embedding neighbourhood of 'man' and 'nurse' with the embedding neighbourhood of 'woman'. The embedding model faithfully captures this distributional asymmetry as a geometric relationship. Importantly, the model does not invent or amplify these biases beyond what is present in the training data — but it does preserve and operationalise them in a form that is directly usable by downstream systems, making the biases actionable in ways that the original text was not.

The propagation of embedding bias into downstream systems is the primary practical concern. A text classification system trained with biased embeddings as features will learn to associate the feature dimensions corresponding to female-associated words with categories that are historically female-dominated, even when those associations are spurious for the specific classification task. A hiring algorithm that uses biased embeddings will systematically penalise CVs containing female-associated language, not because the algorithm was explicitly programmed to discriminate, but because the embedding space it uses encodes occupational gender associations from historical hiring patterns. This is not a hypothetical risk: the Amazon hiring tool case study (2018) directly illustrates how this propagation works in practice.

Case Study: Amazon Hiring Algorithm Bias (2018)

Amazon developed an AI recruitment tool trained on historical hiring data from 2004–2014. Because the technology sector historically employed far more men than women, the training data reflected this imbalance. The model learned to penalise CVs that used language associated with women — including the phrase 'women's chess club captain' and the names of women's colleges — because such language was negatively correlated with hiring outcomes in the historical data. The bias originated in the training corpus, was encoded in the word embeddings used as features, and was amplified by the neural classifier. Amazon scrapped the tool in 2017 after discovering the bias. This case established that embedding bias is not an academic concern but a direct operational risk for AI-powered decision systems.

Debiasing methods have been developed to reduce the gender (and other demographic) bias in pre-trained embeddings without destroying their semantic utility. The 'Hard Debiasing' method of Bolukbasi et al. (2016) operates in two steps: first, identify the 'bias direction' in embedding space — the principal direction that captures the most variance in the difference vectors between gender-paired word sets (man/woman, king/queen, he/she). Second, for all words that should be gender-neutral (occupations, activities, objects), project their embeddings onto the subspace orthogonal to the bias direction, removing the gender component while preserving all other dimensions of the embedding. This approach successfully reduces gender stereotype scores on standard benchmarks while preserving word similarity and analogy performance for non-gender-related tasks.

However, debiasing is not a complete solution, and subsequent research has revealed important limitations. Gonen and Goldberg (2019) showed that hard debiasing reduces the geometric signal of gender bias without eliminating the underlying information: a linear classifier trained on the 'debiased' embeddings can still recover gender associations at above-chance rates, suggesting that the bias is distributed across many dimensions of the embedding space rather than concentrated in a single direction. More fundamentally, removing bias from embeddings does not remove bias from the training data; if a debiased embedding is used to train a classifier on the same biased historical data, the classifier can relearn the bias through other feature pathways. Addressing embedding bias is therefore a necessary but not sufficient step — the broader data collection, annotation, and evaluation practices must also be reformed.

9 Research Frontiers

Probing, interpretability, and dynamic embeddings

The embedding research agenda has expanded substantially beyond the original goals of word similarity and analogy performance. Three active research directions define the current frontiers: probing classifiers for interpretability, dynamic embeddings that track linguistic change over time, and the theoretical unification of different embedding approaches through information-theoretic and algebraic analysis.

Probing classifiers (Conneau et al., 2018; Tenney et al., 2019) are a methodology for understanding what linguistic information is encoded in a representation by training small classifiers to predict specific linguistic properties from the representation. If a probing classifier can accurately predict the part-of-speech tag of a word from its ELMo or BERT representation, this provides evidence that POS information is encoded in that representation. If it cannot, POS information is either absent or non-linearly encoded. Probing studies have revealed a remarkably consistent stratification of linguistic knowledge across the layers of deep language models: surface features (word identity, capitalisation) in lower layers; syntactic features (POS, dependency structure) in middle layers; semantic and pragmatic features (semantic roles, coreference, discourse relations) in upper layers. This stratification holds for LSTM-based models (ELMo) and transformer-based models (BERT) alike, suggesting it reflects a general principle of how hierarchical neural architectures process language.

Dynamic word embeddings extend the distributional hypothesis to the temporal dimension, asking not just what a word means in a given corpus but how its meaning changes over time. Hamilton et al. (2016) trained separate Word2Vec models on decade-by-decade samples of historical English text (Google Books corpus, 1800–2000) and developed methods to align the resulting embedding spaces, enabling the tracking of semantic shifts across centuries. Their analysis confirmed well-known cases of semantic change — 'gay' shifting from 'joyful' to 'homosexual' over the 20th century, 'nice' shifting from 'foolish' to 'pleasant' over five centuries — while also revealing more subtle cultural changes: 'broadcast' shifting from an agricultural term ('scattering seeds') to a communications term as radio technology developed. These methods have applications in historical linguistics, computational social science, and the analysis of how concept meanings evolve in scientific and legal corpora.

The theoretical unification of word embedding methods has been a major intellectual contribution of the post-Word2Vec literature. Levy and Goldberg's (2014) proof that Skip-Gram with Negative Sampling implicitly factorises a Shifted Positive Pointwise Mutual Information (SPPMI) matrix connected prediction-based embeddings (Word2Vec) to count-based embeddings (LSA, GloVe) through a common mathematical framework. PMI — the log ratio of the joint probability of two words co-occurring to the product of their individual probabilities — is the information-theoretic measure of how much more often two words co-occur than chance would predict. The SPPMI matrix is a thresholded, shifted version of the PMI matrix, and its truncated SVD approximation is essentially what Word2Vec computes via stochastic optimisation. This result placed both families of embedding methods on the same theoretical footing and shifted the research agenda from 'which method is better' to 'which approximation to SPPMI factorisation is most efficient and effective for a given application'.

Research Spotlight: Are Embeddings Just Compressed Co-occurrence Statistics? (Levy & Goldberg, 2014)

Levy & Goldberg proved mathematically that Skip-Gram with Negative Sampling is implicitly factorising a shifted Pointwise Mutual Information (PMI) matrix — unifying count-based (LSA, GloVe) and prediction-based (Word2Vec) methods through a common information-theoretic framework. This result demonstrated that the apparently different architectures are solving the same underlying optimisation problem via different numerical methods. It also clarified why 'analogy arithmetic' works: the PMI matrix encodes first-order co-occurrence statistics, and differences

between PMI-based vectors correspond to second-order conditional probability ratios, which are precisely the quantities that encode semantic relationships like gender, tense, and capital-country correspondence.

The intersection of embedding research with large language models raises new questions about what it means to 'represent' word meaning. BERT and GPT-class models do not have a fixed embedding for each word; instead, they produce contextualised representations that integrate information from the entire sentence (and for long-context models, the entire document). Probing studies of BERT reveal that its representations encode substantially richer linguistic information than ELMo or Word2Vec — including complex syntactic phenomena like long-distance dependencies and garden-path structures — raising the question of whether the distinction between 'word meaning' and 'sentence meaning' is a useful one for models that process language through attention over full contexts. The research direction of 'geometric probing' — studying the geometric structure of representation spaces in large transformers rather than the information decodable by linear probes — is currently one of the most active areas at the intersection of linguistics, cognitive science, and deep learning.

Problem Statement

We have three short documents and want to represent them as numerical vectors suitable for machine learning classification. This walkthrough traces every mathematical operation from raw text to a classified prediction, showing exactly how each number is computed.

Corpus

d_1 = "cat sat on mat"
d_2 = "dog sat on log"
d_3 = "cat and dog sat"

Goal: Build a vector representation, apply TF-IDF weighting, compute document similarities, and classify a new document.

WALKTHROUGH STEPS

Step 1 · Vocabulary Construction

The first operation in any BoW pipeline is identifying the set of all unique tokens (words) that appear anywhere in the corpus. Using simple whitespace tokenisation, we split each document on spaces and take the union of the resulting token sets.

Union across all documents

tokens(d_1) = {cat, sat, on, mat}
tokens(d_2) = {dog, sat, on, log}
tokens(d_3) = {cat, and, dog, sat}

$V = \text{tokens}(d_1) \cup \text{tokens}(d_2) \cup \text{tokens}(d_3)$
V = {cat, sat, on, mat, dog, log, and}

$|V| = 7$

We assign each term a fixed integer index. The ordering is arbitrary but must remain consistent throughout the pipeline — every vector and matrix operation depends on all positions meaning the same term.

Index	Term	Appears in documents
0	cat	d_1, d_3
1	sat	d_1, d_2, d_3
2	on	d_1, d_2

3	mat	d_1
4	dog	d_2, d_3
5	log	d_2
6	and	d_3

Key insight: the vocabulary defines the dimensionality of every vector. With |V|=7, each document will be a 7-dimensional vector — one dimension per vocabulary term.

Step 2 · Count Vector Representation

Each document is encoded as a vector v of length |V| where v[i] = count of vocabulary term i in that document. This is a simple frequency count — no weighting yet.

Formal definition

v_j[i] = count(term_i in document_j)

For term_i not in document_j: v_j[i] = 0

Working through each document position by position:

d_1 = "cat sat on mat"

Index 0 (cat) : appears 1 time → $v_1[0] = 1$
Index 1 (sat) : appears 1 time → $v_1[1] = 1$
Index 2 (on) : appears 1 time → $v_1[2] = 1$
Index 3 (mat) : appears 1 time → $v_1[3] = 1$
Index 4 (dog) : appears 0 times → $v_1[4] = 0$
Index 5 (log) : appears 0 times → $v_1[5] = 0$
Index 6 (and) : appears 0 times → $v_1[6] = 0$

$v_1 = [1, 1, 1, 1, 0, 0, 0]$ ✓ verified

d_2 = "dog sat on log"

Index 0 (cat) : appears 0 times → $v_2[0] = 0$
Index 1 (sat) : appears 1 time → $v_2[1] = 1$
Index 2 (on) : appears 1 time → $v_2[2] = 1$
Index 3 (mat) : appears 0 times → $v_2[3] = 0$
Index 4 (dog) : appears 1 time → $v_2[4] = 1$
Index 5 (log) : appears 1 time → $v_2[5] = 1$
Index 6 (and) : appears 0 times → $v_2[6] = 0$

$v_2 = [0, 1, 1, 0, 1, 1, 0]$ ✓ verified

d_3 = "cat and dog sat"

Index 0 (cat) : appears 1 time → $v_3[0] = 1$
Index 1 (sat) : appears 1 time → $v_3[1] = 1$
Index 2 (on) : appears 0 times → $v_3[2] = 0$
Index 3 (mat) : appears 0 times → $v_3[3] = 0$
Index 4 (dog) : appears 1 time → $v_3[4] = 1$
Index 5 (log) : appears 0 times → $v_3[5] = 0$
Index 6 (and) : appears 1 time → $v_3[6] = 1$

$v_3 = [1, 1, 0, 0, 1, 0, 1]$ ✓ verified

Step 3 · Document-Term Matrix

Stacking the three vectors row-by-row produces the document-term matrix X of shape (D × |V|) = (3 × 7). Each row is a document; each column is a vocabulary term.

	cat	sat	on	mat	dog	log	and
d_1	1	1	1	1	0	0	0
d_2	0	1	1	0	1	1	0
d_3	1	1	0	0	1	0	1

Reading the matrix row-wise gives the document vector. Reading column-wise gives the term's distribution across documents — useful for computing document frequency (df) in Step 4.

Why the matrix is sparse

Each document uses only 4 of 7 vocabulary terms, leaving 3 zeros per row. In real corpora the sparsity is extreme: a 50,000-word vocabulary where each document uses ~300 unique words leaves 49,700 zeros per row — sparsity above 99%. This is why scipy.sparse.csr_matrix (which stores only non-zero values and their indices) is used in practice instead of dense NumPy arrays.

Step 4 · Vocabulary Filtering (min_df = 2)

min_df removes terms that appear in fewer than min_df documents. This eliminates hapax legomena (words seen only once) that cannot be reliably estimated and that inflate vocabulary size without contributing predictive power.

Document Frequency formula

$$df(t) = |\{ d \in D : t \in d \}|$$
= number of documents containing term t

Keep term t if $df(t) \geq min\_df$

Computing df for each term in our vocabulary:

Term	Present in	df(t)	min_df=2 ?
cat	d_1, d_3	2	✓ keep
sat	d_1, d_2, d_3	3	✓ keep
on	d_1, d_2	2	✓ keep
mat	d_1 only	1	✗ remove
dog	d_2, d_3	2	✓ keep
log	d_2 only	1	✗ remove
and	d_3 only	1	✗ remove

After filtering, the reduced vocabulary V' = {cat, sat, on, dog} with new consecutive indices 0–3. Dimensionality drops from 7 to 4 — a 43% reduction on this tiny corpus. On a real 50,000-word vocabulary with min_df=5, reductions of 70–90% are typical.

Step 5 · Reduced Document-Term Matrix

Reconstruct vectors keeping only terms in V'. Columns for removed terms (mat, log, and) are dropped; row values for surviving terms are unchanged.

Reconstructed vectors

v'_1 (d_1 = "cat sat on mat"): cat=1 sat=1 on=1 dog=0 → [1, 1, 1, 0] ✓
v'_2 (d_2 = "dog sat on log"): cat=0 sat=1 on=1 dog=1 → [0, 1, 1, 1] ✓
v'_3 (d_3 = "cat and dog sat"): cat=1 sat=1 on=0 dog=1 → [1, 1, 0, 1] ✓

	cat	sat	on	dog
d_1	1	1	1	0
d_2	0	1	1	1
d_3	1	1	0	1

Note: 'mat' is in d_1 and 'log' is in d_2, but their columns are removed. The counts for the remaining terms (cat, sat, on, dog) are unaffected by the removal — we simply project onto the subspace of kept terms.

Step 6 · TF-IDF Transformation

TF-IDF reweights raw counts so that terms that appear in fewer documents receive higher weight — they are more discriminative. A term appearing in every document is useless for classification; a term appearing in only one document is maximally discriminative.

TF-IDF formula used in this walkthrough

TF(t, d) = raw count of term t in document d (from Step 5)
IDF(t) = log(N / df(t)) + 1
where N = total number of documents
and log is the natural logarithm (base e)

TF-IDF(t,d) = TF(t,d) × IDF(t)

The +1 in IDF ensures IDF ≥ 1 for all terms.
Terms in every document get IDF = log(1)+1 = 1 (not zero).

Computing IDF for each term in V' (N=3 documents):

IDF calculations (natural log, 4 decimal places)

IDF(cat) = log(3/2) + 1 = log(1.5) + 1 = 0.4055 + 1 = 1.4055 ≈ 1.405 ✓
IDF(sat) = log(3/3) + 1 = log(1.0) + 1 = 0.0000 + 1 = 1.0000 ✓
IDF(on) = log(3/2) + 1 = log(1.5) + 1 = 0.4055 + 1 = 1.4055 ≈ 1.405 ✓
IDF(dog) = log(3/2) + 1 = log(1.5) + 1 = 0.4055 + 1 = 1.4055 ≈ 1.405 ✓

Intuition: 'sat' appears in all 3 documents → lowest IDF = 1.000
cat/on/dog appear in exactly 2 of 3 → higher IDF = 1.405

Now multiply each TF value by its IDF:

TF-IDF weights

$d_1 = [1, 1, 1, 0]$: $[1\times1.405,\ 1\times1.000,\ 1\times1.405,\ 0\times1.405] = [1.405, 1.000, 1.405, 0.000]$ ✓
$d_2 = [0, 1, 1, 1]$: $[0\times1.405,\ 1\times1.000,\ 1\times1.405,\ 1\times1.405] = [0.000, 1.000, 1.405, 1.405]$ ✓
$d_3 = [1, 1, 0, 1]$: $[1\times1.405,\ 1\times1.000,\ 0\times1.405,\ 1\times1.405] = [1.405, 1.000, 0.000, 1.405]$ ✓

	cat	sat	on	dog
d_1	1.405	1.000	1.405	0.000
d_2	0.000	1.000	1.405	1.405
d_3	1.405	1.000	0.000	1.405

What TF-IDF reveals here

All three documents share 'sat' (IDF=1.000, the minimum). This makes 'sat' the least informative term for distinguishing documents — exactly what TF-IDF should encode. The unique terms 'cat', 'on', 'dog' each appear in exactly 2 of 3 documents and receive the higher IDF of 1.405, making them better discriminators. If any term had appeared in only 1 document (like 'mat', 'log', 'and' before filtering), it would receive IDF = log(3/1)+1 ≈ 2.099 — the highest possible weight.

Step 7 · Cosine Similarity

Cosine similarity measures the angle between two vectors in the shared feature space. It ranges from 0 (perpendicular — no shared features) to 1 (parallel — identical direction). It is preferred over Euclidean distance for text because it is length-normalised: a short document and a long document on the same topic should be judged similar.

Cosine similarity formula

$$\text{sim}(d_i, d_j) = (d_i \cdot d_j) / (\|d_i\| \times \|d_j\|)$$

where: $d_i \cdot d_j$ = dot product $= \Sigma_k\, d_i[k] \times d_j[k]$
$\|d\|$ = Euclidean norm $= \sqrt{(\Sigma_k\, d[k]^2)}$

Using the count vectors from Step 5: $v'_1=[1,1,1,0]$, $v'_2=[0,1,1,1]$, $v'_3=[1,1,0,1]$

sim(d_1, d_2) — step by step

Dot product: $v'_1 \cdot v'_2 = (1\times0) + (1\times1) + (1\times1) + (0\times1) = 0 + 1 + 1 + 0 = 2$
$\|v'_1\| = \sqrt{(1^2 + 1^2 + 1^2 + 0^2)} = \sqrt{3} = 1.732$
$\|v'_2\| = \sqrt{(0^2 + 1^2 + 1^2 + 1^2)} = \sqrt{3} = 1.732$
$\text{sim}(d_1,d_2) = 2 / (1.732 \times 1.732) = 2 / 3 = 0.667$ ✓

sim(d_1, d_3) — step by step

Dot product: $v'_1 \cdot v'_3 = (1\times1) + (1\times1) + (1\times0) + (0\times1) = 1 + 1 + 0 + 0 = 2$
$\|v'_1\| = \sqrt{3} = 1.732$
$\|v'_3\| = \sqrt{(1^2 + 1^2 + 0^2 + 1^2)} = \sqrt{3} = 1.732$
$\text{sim}(d_1,d_3) = 2 / (1.732 \times 1.732) = 2 / 3 = 0.667$ ✓

sim(d_2, d_3) — step by step

Dot product: $v'_2 \cdot v'_3 = (0\times1) + (1\times1) + (1\times0) + (1\times1) = 0 + 1 + 0 + 1 = 2$

$||v'_2||$ $= \sqrt{3} = 1.732$
$||v'_3||$ $= \sqrt{3} = 1.732$
$\text{sim}(d_2,d_3) = 2 / (1.732 \times 1.732) = 2 / 3 = 0.667$ ✓

What equal similarities reveal about BoW

All three pairwise similarities are identically 0.667. This is not coincidence — it is a structural property of this particular corpus: every pair of documents shares exactly 2 of the 4 kept terms, and all three vectors have the same L2 norm ($\sqrt{3}$). The equal similarities illustrate a core limitation of count vectors: they cannot distinguish which shared terms matter more. The TF-IDF matrix (Step 6) assigns different weights to different terms, so TF-IDF-based similarities would differ across pairs. Try computing $\text{sim}(d_1,d_2)$ on the TF-IDF vectors as a practice exercise.

Step 8 · Naïve Bayes Classification

Naïve Bayes is a probabilistic classifier that applies Bayes' theorem with the 'naïve' conditional independence assumption: given the class label, each feature (word) is generated independently. Despite this unrealistic assumption, it works well in practice for text classification and is still widely used as a fast, interpretable baseline.

Bayes' theorem for text classification

$P(y \mid d) \propto P(y) \times P(d \mid y)$ (posterior ∝ prior × likelihood)

Naïve independence assumption:
$P(d \mid y) = \prod_k P(t_k \mid y)^{\{tf(t_k,d)\}}$

Only terms with tf > 0 contribute to the product ($t^0 = 1$).
In log space: $\log P(y|d) \propto \log P(y) + \Sigma_k \, tf(t_k,d) \times \log P(t_k|y)$

Labels: d_1, d_2 are class A (y=0); d_3 is class B (y=1). Using the reduced vocabulary V' = {cat, sat, on, dog}.

Prior probabilities

$P(A) = P(y=0) = |\{d_1,d_2\}| / |D| = 2/3 \approx 0.667$
$P(B) = P(y=1) = |\{d_3\}| / |D| = 1/3 \approx 0.333$

Laplace smoothing prevents zero probabilities for terms not seen in a class. With smoothing parameter α=1:

Laplace-smoothed conditional probability

$P(t \mid y) = (\text{count}(t \text{ in class } y) + \alpha) / (\text{total words in class } y + \alpha \times |V'|)$

where $|V'| = 4$ (size of reduced vocabulary)

Class A (d_1=[1,1,1,0] + d_2=[0,1,1,1])

Total tokens in A = $\text{sum}(v'_1) + \text{sum}(v'_2) = 3 + 3 = 6$ ✓

Count of each term in A:
cat : 1 (from d_1) + 0 (from d_2) = 1
sat : 1 + 1 = 2
on : 1 + 1 = 2
dog : 0 + 1 = 1

Denominator for Laplace: total_A + α×|V'| = 6 + 1×4 = 10

P(cat|A) = (1+1)/10 = 2/10 = 0.200 ✓
P(sat|A) = (2+1)/10 = 3/10 = 0.300 ✓
P(on |A) = (2+1)/10 = 3/10 = 0.300 ✓
P(dog|A) = (1+1)/10 = 2/10 = 0.200 ✓

Class B (d_3=[1,1,0,1])

Total tokens in B = sum(v'_3) = 1+1+0+1 = 3 ✓

Count of each term in B:
cat : 1 sat : 1 on : 0 dog : 1

Denominator for Laplace: total_B + α×|V'| = 3 + 1×4 = 7

P(cat|B) = (1+1)/7 = 2/7 ≈ 0.286 ✓
P(sat|B) = (1+1)/7 = 2/7 ≈ 0.286 ✓
P(on |B) = (0+1)/7 = 1/7 ≈ 0.143 ✓
P(dog|B) = (1+1)/7 = 2/7 ≈ 0.286 ✓

Classifying a new document d_4 = "cat sat" → vector [1, 1, 0, 0]:

Classification of d_4

Only terms with count > 0 contribute to the product; 0-count terms give t^0=1.

P(A | d_4) ∝ P(A) × P(cat|A)^1 × P(sat|A)^1
= 0.667 × 0.200 × 0.300
= 0.667 × 0.060
= 0.040 ✓

P(B | d_4) ∝ P(B) × P(cat|B)^1 × P(sat|B)^1
= 0.333 × 0.286 × 0.286
= 0.333 × 0.082 (0.286² = 0.0816 ≈ 0.082 — rounding)
= 0.027 ✓

0.040 > 0.027 → classify d_4 as class A ✓

The unnormalised scores (0.040 and 0.027) can be converted to proper posterior probabilities by normalising: P(A|d_4) = 0.040 / (0.040+0.027) ≈ 0.597, P(B|d_4) ≈ 0.403. The classification remains A.

Step 9 · Sparsity Analysis ⚠ Error Corrected

Sparsity quantifies what fraction of the document-term matrix cells contain zeros. High sparsity motivates sparse storage formats (scipy.sparse) that dramatically reduce memory usage by storing only non-zero entries.

Sparsity formula

Sparsity = 1 - (non-zero entries) / (total cells)
= 1 - (non-zero entries) / (D × |V|)

Verified count:

Original 3×7 matrix — non-zero entries

d_1 = [1, 1, 1, 1, 0, 0, 0] → 4 non-zero (cat, sat, on, mat)
d_2 = [0, 1, 1, 0, 1, 1, 0] → 4 non-zero (sat, on, dog, log)
d_3 = [1, 1, 0, 0, 1, 0, 1] → 4 non-zero (cat, sat, dog, and)

Total non-zero = 4 + 4 + 4 = 12
Total cells = 3 × 7 = 21
Sparsity = 1 - 12/21 = 1 - 0.571 = 0.429 (42.9% zeros) ✓

(The original document incorrectly stated 11 non-zero / 47.6% sparsity)

Reduced 3×4 matrix — non-zero entries

v'_1 = [1, 1, 1, 0] → 3 non-zero
v'_2 = [0, 1, 1, 1] → 3 non-zero
v'_3 = [1, 1, 0, 1] → 3 non-zero

Total non-zero = 3 + 3 + 3 = 9
Total cells = 3 × 4 = 12
Sparsity = 1 - 9/12 = 1 - 0.75 = 0.25 (25.0% zeros) ✓

The min_df filtering reduced sparsity from 42.9% to 25.0% by removing the three terms that each appeared in only one document — which were, by construction, the sparsest columns. This is a general property: min_df filtering preferentially removes the sparsest columns, improving data density.

For comparison, here are typical real-world sparsity figures:

Scenario	Typical sparsity
20 Newsgroups (18,000 docs, 130,000 vocab)	~99.5%
Reuters-21578 (10,000 docs, 50,000 vocab)	~99.8%
After min_df=10 filtering (Reuters)	~98.5%
Our toy corpus (3 docs, 7 vocab, raw)	42.9%
Our toy corpus (3 docs, 4 vocab, min_df=2)	25.0%

A Formula Summary

All mathematical operations used in this walkthrough

The following table collects every formula used, its symbolic definition, and the step where it was applied.

Formula / Operation	Definition
1. Vocabulary	$V = \cup_j \text{tokens}(d_j)$
2. Count vector	v_j[i] = count(t_i in d_j)
3. Document frequency	df(t) = |{d ∈ D : t ∈ d}|

4. min_df filter	Keep t if df(t) ≥ min_df
5. max_df filter	Keep t if df(t) ≤ max_df × N
6. IDF (with +1 smoothing)	idf(t) = log(N / df(t)) + 1
7. TF-IDF weight	w(t,d) = tf(t,d) × idf(t)
8. Cosine similarity	$sim(d_i,d_j) = (d_i \cdot d_j) / (\|\|d_i\|\| \times \|\|d_j\|\|)$
9. Naïve Bayes posterior	$P(y\|d) \propto P(y) \times \prod P(t\|y)^{tf(t,d)}$
10. Laplace smoothing	P(t|y) = (c(t,y)+α) / (c(y)+α|V'|)
11. Sparsity	1 − (non-zero entries) / (D × |V|)
12. Compression ratio	Dense size / sparse size = |V| / u_avg

Notation key

Symbol	Meaning
D	set of all documents in the corpus
N	total number of documents |D|
V	vocabulary (set of all unique terms)
|V|	vocabulary size
t, t_i	a term (word) in the vocabulary
d, d_j	a document
tf(t,d)	term frequency of t in d
df(t)	document frequency of t
α	Laplace smoothing parameter (typically 1)
u_avg	average unique words per document

KEY TAKEAWAYS

- BoW: every document becomes a count vector of length |V|. Word order is discarded; only token presence and frequency are captured.
- min_df filtering removes rare tokens that cannot be reliably estimated. It reduces dimensionality without significant loss of discriminative power.
- TF-IDF reweights counts by IDF = log(N/df)+1. Ubiquitous terms ('sat' in all 3 docs) get IDF=1; terms in fewer documents get higher IDF — they are more discriminative.
- Cosine similarity = dot product divided by the product of norms. It is length-normalised, making short and long documents comparable on topic.
- Naïve Bayes classifies by multiplying prior × per-term likelihoods. Laplace smoothing (α=1) prevents zero probabilities for unseen terms.
- Sparsity correction: the original document counted 11 non-zero entries in the 3×7 matrix. The correct count is 12 (each document has 4 non-zero tokens). Correct sparsity = 42.9%, not 47.6%.
- All other calculations (Steps 1–8 and the reduced-matrix sparsity in Step 9) are fully correct.